Year 2026–2027 Semester V NEP

Course Agile Software Development and DevOps Lab

Class T1 Date of Submission 13/10/2026

A Mini Project Report on

Helpdesk

A Full Stack Support Ticket Portal Delivered Through an Agile and DevOps Workflow

Submitted in partial fulfilment of the requirements of the course
Agile Software Development and DevOps Lab
Semester V NEP · Academic Year 2026–2027 · Class T1

Submitted by

Name	Roll No.
Atharva Gokhale	2401031
Utsav Golani	2401032
Janhavi Khadse	2401046

Under the guidance of

Prof. Sanober Sheikh

Department of Artificial Intelligence and Data Science

Thadomal Shahani Engineering College

2026–2027

CERTIFICATE

This is to certify that the mini project entitled “Helpdesk: A Full Stack Support Ticket Portal Delivered Through an Agile and DevOps Workflow” is the work submitted by Atharva Gokhale (2401031), Utsav Golani (2401032) and Janhavi Khadse (2401046), in partial fulfilment of the requirements of the course Agile Software Development and DevOps Lab, Semester V NEP, during the academic year 2026–2027.

The project presents a support ticket application together with version control, containerization, multi-service deployment, continuous integration, automated deployment, Agile task tracking and monitoring.

_____

Project Guide

Prof. Sanober Sheikh

_____

Head of Department

Date: _____

Place: _____

ABSTRACT

Helpdesk is a full-stack support ticket portal developed to demonstrate the seven experiments of the Agile Software Development and DevOps laboratory within one connected software delivery workflow. The application allows requesters to raise support tickets and follow their progress, while agents and administrators manage priorities, assignments, comments and ticket status. The interface also provides a queue overview, activity information and team management.

The frontend is implemented using React and served through nginx. A Python Flask backend implements the application API, authentication and role-based access checks, while PostgreSQL stores the deployed application data. Docker packages the frontend and backend into separate images, and Docker Compose coordinates these services with a persistent database. Production and staging are deployed separately on the same local Docker engine.

Git and GitHub maintain the source code, branches, pull requests and release tag. Jenkins reads the Jenkinsfile from the repository, executes 32 automated backend tests, builds Docker images and deploys eligible branches. A smoke-test gate verifies the deployed application and attempts automatic restoration of the previous images if a candidate fails. A staging rehearsal verified database persistence, failed-candidate rollback and recovery after a controlled backend outage.

Jira records the development work in the HDEV project with an epic, nine stories and an active sprint. Prometheus collects backend and cAdvisor metrics, and Grafana displays request rate, latency, availability and resource measurements. On the demonstrated Docker Desktop environment, resource panels explicitly use Docker host totals where individual container series are unavailable. The project connects application development, delivery and operation through observable evidence rather than treating each tool as an isolated exercise.

Keywords: Agile, Helpdesk, Git, GitHub, Docker, Docker Compose, Jenkins, CI/CD, Jira, Prometheus, Grafana, React, Flask, PostgreSQL.

CONTENTS

The report follows the implementation from application requirements to delivery, verification and operation.

Section	Page
1 Introduction	5
2 Requirements and Agile Planning	7
3 System Design	9
4 Application Implementation	12
5 DevOps Implementation	14
6 Testing	25
7 Implementation	27
8 Results and Discussion	29
9 Conclusion and Future Scope	30
A Appendix Running the Project	31

1 Introduction

1.1 Background

Support requests are often scattered across conversations or informal lists. This makes it difficult to identify responsibility, determine priority and understand whether a request has been resolved. A shared support portal provides a persistent record of the request, its owner, status and discussion.

A useful application also requires a dependable way to deliver changes. Manual installation and deployment can create differences between environments and make failures difficult to reproduce. This project applies the laboratory experiments to the same website so that code changes, testing, deployment and monitoring form one connected workflow.

1.2 Problem Statement

To develop a full-stack support ticket portal and deliver it through an integrated Agile and DevOps workflow in which the source is version-controlled, the application is containerized, services are coordinated, changes are automatically tested and deployed, project work is tracked in Jira, and application health and performance are monitored.

1.3 Objectives

The application objectives are to provide authenticated requester and staff workflows, persistent tickets, priorities, assignments, comments, status changes and a dashboard. The delivery objectives are to maintain a traceable Git history, build repeatable Docker images, separate staging from production, run automated tests in Jenkins, verify deployments through smoke tests and recover from a failed candidate.

The operational objectives are to expose application metrics, collect them with Prometheus, display them in Grafana, demonstrate a backend availability alert and record verified project completion in Jira.

1.4 Scope

The implemented scope is a local academic deployment of the support portal and its seven DevOps experiments. The portal is not a public hosted service. Email delivery, password reset, distributed rate limiting, cloud infrastructure, Kubernetes and automatic synchronization of portal tickets to Jira are outside the demonstrated scope.

1 Introduction continued

1.5 Mapping of the Seven Experiments

Exp.	Laboratory topic	Application in Helpdesk
1	Git and GitHub	main/develop branches, feature branch, two merged pull requests, real conflict exercise and release tag.
2	Docker	Separate multi-stage frontend and backend images, non-root runtime users and health checks.
3	Docker Compose	Frontend, backend and PostgreSQL with networks, persistent volumes and separate environments.
4	Jenkins CI	Pipeline as code, automated backend tests, image builds, JUnit results and archived reports.
5	Automated deployment	GitHub polling, branch-specific deployment, authenticated smoke gate and automatic image rollback.
6	Jira Agile workflow	HDEV epic, nine stories, active sprint, statuses and an example owner, estimate and acceptance criteria.
7	Prometheus and Grafana	Backend/cAdvisor scraping, five dashboard queries and BackendDown alert recovery.

Table 1.1 Mapping of laboratory experiments to the implemented project.

1.6 End to End Connection

GitHub source code → Jenkins pipeline

Docker images → Compose deployment → Helpdesk website

Backend metrics and cAdvisor → Prometheus → Grafana

Jira → development planning and task tracking

The first six components exchange source, artifacts, application traffic or monitoring data. Jira supports project tracking; its issues are not the same database records as support tickets created inside the website.

2 Requirements and Agile Planning

2.1 User Roles

Role	Responsibility
Requester	Register, sign in, create and view permitted tickets, and participate in their ticket conversations.
Agent	Work with the support queue, update permitted ticket properties, assign work and reply to requests.
Administrator	Perform staff workflows and create agent accounts through the Team interface.

2.2 Functional Requirements

The application shall authenticate users using email and password, maintain a session and enforce requester, agent and administrator permissions. Requesters shall be restricted to their own tickets. Public registration shall create requester accounts, while agent creation shall require administrator authorization.

Tickets shall contain a title, description, category, priority, requester, optional assignee and status. Users shall be able to search and filter the queue, open a ticket, add comments and view its activity history. Authorized staff shall be able to change assignments and progress tickets through open, in progress, waiting and resolved states.

The dashboard shall summarize active, unassigned and overdue work, ticket activity and priority distribution using the application database. The Team interface shall show staff and workload information.

2.3 Non Functional Requirements

The application shall run in consistent container environments. Database storage shall survive application recreation. Source changes shall be tested before deployment, deployment results shall be checked through the API, and failed candidates shall trigger recovery of previously running images. Sensitive environment values shall remain outside Git and be supplied to Jenkins through credentials.

Health checks shall expose readiness, and application metrics shall support operational monitoring. Local published ports shall bind to loopback for the classroom deployment.

2 Requirements and Agile Planning continued

2.4 Actual Jira Backlog

The Jira project is Helpdesk DevOps, key HDEV. Epic HDEV-3 is “CI/CD and Monitoring of a Full-Stack Support Ticket Portal”. HDEV Sprint 1 was started on 12 October 2026. The following statuses record the verified presentation state.

Issue	Work item	Status
HDEV-1	Git branches, pull requests and release evidence	Done
HDEV-2	Portal accounts, tickets, assignments and comments	Done
HDEV-4	Docker images and health checks	Done
HDEV-5	Compose isolation and persistent PostgreSQL	Done
HDEV-6	Jenkins tests and image builds	Done
HDEV-7	Polling, smoke gate and rollback	Done
HDEV-8	Monitoring and outage recovery	Done
HDEV-9	Agile board and acceptance criteria	Done
HDEV-10	Classroom review and presentation	In Progress

2.5 Agile Traceability

A Jira story describes the development work. The corresponding implementation is stored in GitHub and verified through Jenkins and runtime evidence. HDEV-9 demonstrates an assignee, a two-point estimate, acceptance criteria and the parent epic. Completion statuses were updated after the technical checks passed.

The sprint is a current academic demonstration. Historical multi-week Scrum progress and burndown are not claimed. Estimates added during the demonstration are not represented as original planning estimates. GitHub references in Jira are explicit links; automatic Jira development integration is not configured.

3 System Design

3.1 Application Architecture

The browser loads the React application from nginx. Requests under /api are proxied by nginx to Flask on the Compose network. Flask validates the session, CSRF token, role and request fields before accessing PostgreSQL through SQLAlchemy. The response is returned as JSON and rendered by React.

Browser

↓ HTTP on localhost 8080 or 8081

React static files and nginx reverse proxy

↓ /api requests on the Compose network

Flask API running under Gunicorn

↓ SQLAlchemy and PostgreSQL driver

PostgreSQL database in a named Docker volume

3.2 Delivery Architecture

Jenkins runs in its own Docker Compose project and connects to the GitHub repository through a multibranch job. It determines the deployment environment from the branch. The pipeline creates image tags using the source commit and build number, making a deployed artifact traceable to a specific build.

main branch → tests → images → production on port 8080

develop branch → tests → images → staging on port 8081

feature branch → tests → images only

3.3 Monitoring Architecture

Prometheus accesses backend:8000/metrics and cadvisor:8080 through the staging network and also scrapes itself. Grafana queries the Prometheus datasource. The monitoring configuration is packaged into dedicated images so Jenkins can deploy it without relying on host-relative configuration mounts.

3 System Design continued

3.4 Technology Stack

Layer	Technology	Purpose
Frontend	React, Vite, nginx	Interactive interface, production assets and API proxy.
Backend	Python, Flask, Gunicorn	API, authentication, role checks and application logic.
Persistence	SQLAlchemy, PostgreSQL	Persistent users, tickets, comments and activity.
Tests	pytest, SQLite	Isolated automated backend test execution.
Version control	Git and GitHub	History, branches, merges and released code.
Runtime	Docker and Compose	Images, services, networks, health and volumes.
Automation	Jenkins and Groovy	CI/CD stages, credentials, reports and recovery.
Planning	Jira	Epic, stories, sprint and work status.
Monitoring	Prometheus, cAdvisor, Grafana	Metrics collection, alerts and dashboards.

3.5 Services and Access Points

Service	Local access	Responsibility
Production frontend	localhost:8080	User-facing production website.
Staging frontend	localhost:8081	Separate development deployment.
Backend	localhost:8000 / 8001	Production / staging health and metrics.
Jenkins	localhost:8088	Multibranch delivery pipeline.
Prometheus	localhost:9090	Targets, queries and alert state.
Grafana	localhost:3000	Provisioned monitoring dashboard.
cAdvisor	localhost:8082	Docker host/container metric exporter.

PostgreSQL is reached internally by service name. Production and staging use separate Compose projects and database volumes.

3 System Design continued

3.6 Database Design

Table	Important fields	Relationship
support_users	id, name, email, password_hash, role	User identity referenced by tickets and comments.
support_tickets	title, description, status, priority, category, requester_id, assignee_id, due_at	Requester required; assignee optional.
support_comments	ticket_id, author_id, body, created_at	Each comment belongs to a ticket and a user.
support_activity	ticket_id, actor_id, message, created_at	Records ticket actions by an authenticated user.

The application uses additive support_* tables, preserving legacy data rather than deleting earlier tables. Comments and activity are related to their parent ticket. A named PostgreSQL volume separates persistent data from replaceable application containers.

3.7 Security and Health Design

Passwords are hashed before storage. Session cookies are HTTP-only and use SameSite=Lax. API mutation requests require the X-CSRF-Token header. Requester access is scoped to the authenticated user, and administrator checks protect agent creation. Environment-specific session keys and database credentials are kept in ignored files and Jenkins secret-file credentials.

The backend /health endpoint performs a database query. The frontend /health response checks nginx availability; by itself it does not prove that the API works. Therefore deployment validation includes an authenticated smoke test through the frontend, exercising registration, session handling and ticket operations.

3.8 Operational Boundaries

This is a trusted local laboratory environment. Jenkins has Docker access and must be restricted to trusted project contributors. Image rollback does not restore a previous database schema or recover a deleted database volume; compatible schema changes are assumed.

4 Application Implementation

4.1 Requester Workflow

Register or sign in → create ticket → view own queue

Open ticket → add comment → follow status and resolution

Registration accepts a name, email and password and creates a requester account. The requester can submit a title and description, choose a category and priority, and follow the request from its ticket detail page. Requester isolation is enforced in the backend rather than depending only on hidden frontend controls.

4.2 Staff Workflow

Sign in → inspect queue → assign work → reply

Update status → resolve ticket → review dashboard

Agents and administrators manage the support queue. Ticket details show the description, conversation, activity and editable properties. Administrators can create agent accounts through the Team interface. The user interface also includes ticket filters, assigned work, reports and a guide.

4.3 Status and Priority Rules

Property	Values / behaviour
Status	open, in_progress, waiting, resolved
Priority	low, medium, high, urgent
Resolution target	Low 72 hours; medium 48; high 24; urgent 4.
Categories	General, Account access, Hardware, Software, Network, Billing
Overdue	A non-resolved ticket whose due_at time has passed.

Resolution targets are application tracking values. The current project does not claim contractual service-level guarantees or external notification delivery.

4 Application Implementation continued

4.4 Repository Structure

```
helpdesk-devops/
  backend/       Flask API, administrator setup and tests
  frontend/      React interface, nginx and Dockerfile
  jenkins/       Jenkins image, plugins and bootstrap
  monitoring/    Prometheus rules and Grafana provisioning
  scripts/       Deploy, rollback, smoke, load and rehearsal
  docs/          Setup, experiment mapping and verification
  Jenkinsfile    Pipeline stages and branch deployment rules
  docker-compose.yml
  docker-compose.monitoring.yml
  .env.example   Names of required private settings
```

4.5 API Responsibilities

Endpoint group	Responsibility
/api/session	Current user and CSRF token.
/api/auth/*	Registration, login, logout and account operations.
/api/tickets	Ticket creation, listing and permitted updates.
/api/tickets/<id>/comments	Ticket conversation.
/api/team	Staff directory and administrator agent creation.
/api/dashboard	Counts, distributions and recent activity.
/health and /metrics	Readiness and Prometheus metrics.

4.6 Frontend and Backend Connection

React calls the API through the same origin. nginx forwards these calls to the backend service, avoiding a separate public API origin for the user interface. The production frontend is built from the committed package lockfile. Flask runs under Gunicorn rather than the development server.

5 DevOps Implementation

5.1 Experiment 1 Version Control

Git maintains the project history and GitHub hosts the private repository gokhale-atharva/helpdesk-devops. main represents production and develop represents staging. A feature branch was used for the Jenkins checkout correction; pull request #1 merged that correction into develop, and pull request #2 promoted it to main.

The initial Jenkins checkout problem was resolved by reading the actual checked-out commit using git rev-parse HEAD before constructing the image tag. This links the pipeline correction to an identifiable commit and pull request rather than changing the pipeline only in the Jenkins interface.

5.1.1 Conflict Exercise and Release Tag

The isolated exercise/git-conflict-resolved branch records a genuine merge conflict created by editing the same content on two practice branches. The conflict markers were inspected and the intended combined content was committed. The merge commit 14f149d preserves both edits. This exercise was separate from the deployment branches.

The annotated tag v1.0.0-demo identifies commit 8545cda, which contains the verified recovery rehearsal. Later documentation and dashboard-label changes are recorded in commit 97412f3. A tag identifies a fixed version; a branch continues to receive new commits.

```
git branch -a
git log --oneline --graph --decorate
git show 14f149d
git show v1.0.0-demo
git remote -v
```

These commands inspect branches, merge history, the conflict resolution, the release tag and the remote. GitHub pull requests preserve the proposed changes and merge record; independent teammate approval is not claimed.

5 DevOps Implementation continued

5.1.2 GitHub Evidence

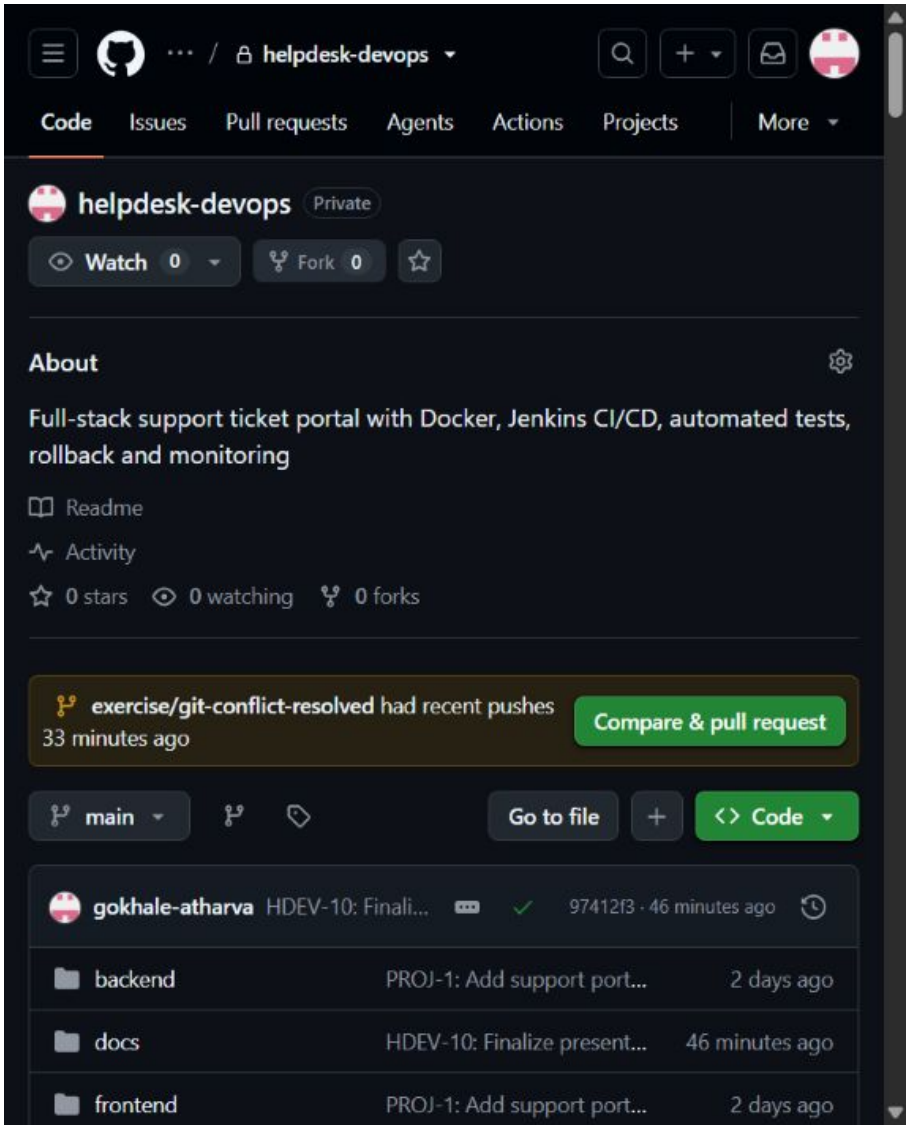


Figure 5.1 Actual Helpdesk GitHub repository showing the main branch and project source.

Repository: https://github.com/gokhale-atharva/helpdesk-devops

The repository contains the application and its delivery configuration together. The private repository requires an authorized GitHub account when presented from another browser.

5 DevOps Implementation continued

5.2 Experiment 2 Docker Containerization

Two multi-stage Dockerfiles package the application. The backend builder creates Python dependency wheels. The runtime stage installs these wheels, copies the Flask application and starts Gunicorn as user 10001. The frontend builder runs npm ci and compiles React. Its runtime uses unprivileged nginx, serves the compiled assets and forwards API traffic to Flask.

Image	Build/runtime configuration
Backend	python:3.12.8-slim-bookworm; wheel builder; Gunicorn on port 8000; one worker and four threads.
Frontend	node:22.14.0-alpine3.21 builder; nginxinc/nginx-unprivileged:1.27.4-alpine runtime; port 8080.
Image identity	Commit SHA prefix plus Jenkins build number.
Health	Backend performs a database query; frontend checks its HTTP health route.

5.2.1 Build Commands

```
docker build -t helpdesk-backend:demo backend
docker build -t helpdesk-frontend:demo frontend
docker images
docker ps
docker logs <container-name>
```

The first two commands reproduce the image builds from the repository root. Running the backend also requires database and secret configuration; Compose supplies these dependencies in the demonstrated deployment.

5.2.2 Docker Evidence

Jenkins console logs record the Docker build steps, resulting image tags and subsequent container deployment. The existing todo- image and Compose project prefixes were retained to preserve earlier data and deployment state; the running application is the Helpdesk support portal.

5 DevOps Implementation continued

5.3 Experiment 3 Docker Compose

docker-compose.yml defines frontend, backend and db. The frontend communicates with the backend by service name, and the backend uses PostgreSQL on the internal network. Health-dependent startup ensures that dependent services wait for readiness. Named volumes keep database data separate from the replaceable application containers.

Compose project	Services	Purpose
todo-production	frontend, backend, db	Production website at localhost:8080.
todo-staging	frontend, backend, db	Staging website at localhost:8081.
Staging monitoring overlay	prometheus, grafana, cadvisor	Monitoring on the staging network.
todo-ci	jenkins	Automation server at localhost:8088.

5.3.1 Validation and Deployment

```
docker compose -p todo-staging --env-file .env.staging config --quiet
docker compose -p todo-staging --env-file .env.staging ps
docker compose -p todo-production --env-file .env.production ps
```

Jenkins uses these project identities when deploying. Configuration values are supplied through environment files, and Jenkins obtains the environment file from its credentials store. Production and staging can coexist without sharing their database volumes or frontend ports.

5.3.2 Persistence Verification

The staging rehearsal inserted a dedicated marker into PostgreSQL, restarted the database container, verified that the marker remained and removed the probe row. The recorded result was PASS. This demonstrates persistence across a container restart; it is not a backup and disaster-recovery test.

5 DevOps Implementation continued

5.4 Experiment 4 Jenkins Continuous Integration

The Jenkins multibranch job helpdesk reads Jenkinsfile from GitHub. A periodic repository scan detects changes. The checkout stage selects the current branch and calculates the image tag from the checked-out commit and build number. The dependency stage prepares a virtual environment inside Jenkins, and pytest runs the backend tests there.

Stage	Action
Checkout	Retrieve source and identify commit/environment.
Install deps	Create Python environment and install test dependencies.
Unit tests	Execute pytest and produce JUnit XML.
Build images	Build backend and frontend Docker images.
Deploy	main and develop only; environment lock and credentials.
Smoke test	Display the result of the deployment smoke gate.
Monitoring	Provision monitoring for develop.
Recovery rehearsal	Optional controlled exercise on develop only.

JUnit results and reports/* artifacts are published after the pipeline run. Feature branches build and test without deployment credentials. Environment locks prevent overlapping deployments to the same environment.

5.4.1 CI Evidence

The backend suite contains 32 passing tests in the recorded successful build. The tests run in the Jenkins environment, while the deployed application runs in its own Python Docker image. Runtime compatibility is additionally checked by the authenticated smoke test against PostgreSQL-backed containers. Email notification parameters exist, but SMTP delivery is not part of the demonstrated configuration.

5 DevOps Implementation continued

5.4.2 Jenkins Build Evidence

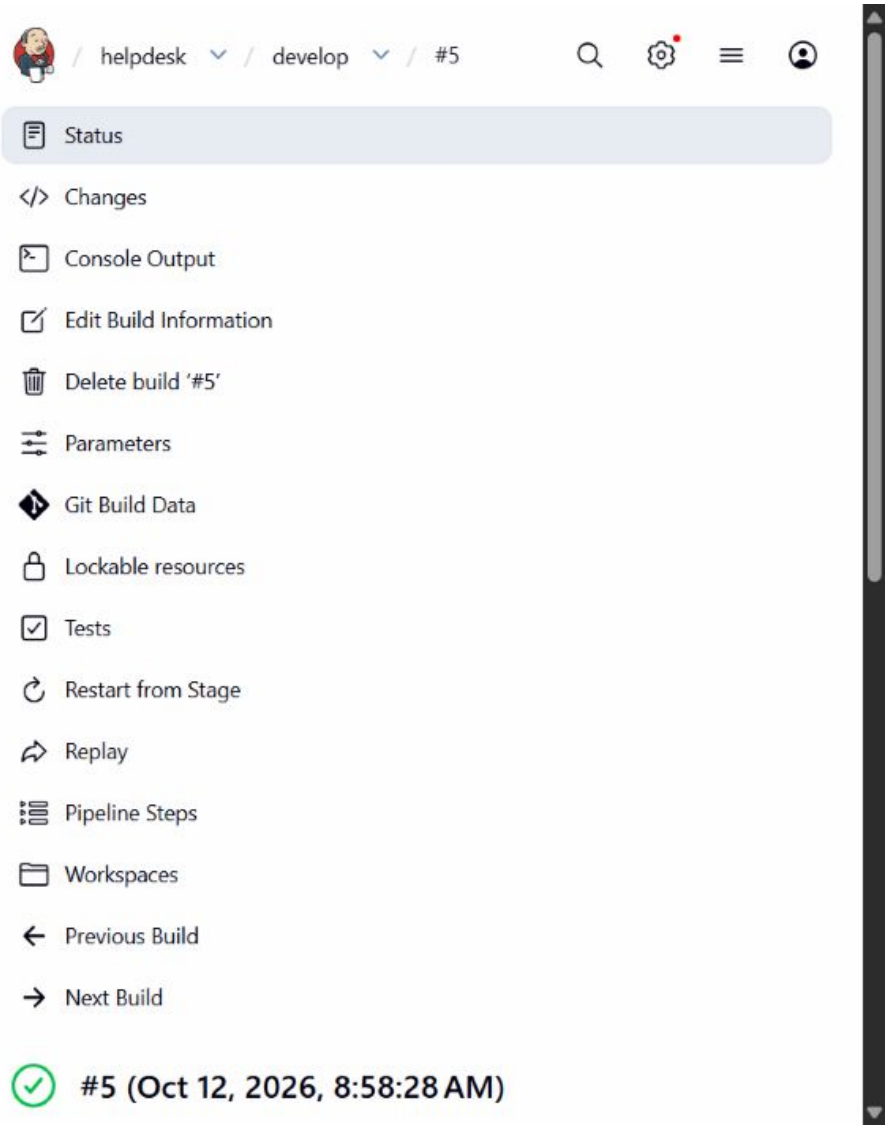


Figure 5.2 Actual Jenkins develop build 5 with archived verification reports.

Evidence page: http://localhost:8088/job/helpdesk/job/develop/5/

This build stores the test report, smoke report, rehearsal output and alert snapshots. Its Git revision links the executed pipeline to source code in GitHub.

5 DevOps Implementation continued

5.5 Experiment 5 Automated Deployment

GitHub repository polling is used for this local demonstration. No public webhook relay is required. Changes to main are eligible for production deployment, changes to develop for staging, and feature branches for build-only verification. The user selected polling rather than a public webhook.

Git push → GitHub scan → Jenkins checkout

Tests pass → build images → choose environment

Snapshot running image IDs → deploy candidate

Authenticated smoke passes → retain candidate

Smoke fails → restore previous images → report failure

5.5.1 Smoke Gate

The smoke test runs through the frontend and checks registration, session/CSRF handling, ticket create/read/update/delete operations, comments, dashboard and requester permission boundaries. Temporary test accounts are cleaned up. This catches failures that a simple HTTP response from nginx cannot detect.

5.5.2 Automatic Rollback

deploy.sh snapshots the actual running frontend and backend image IDs. If deployment or the smoke gate fails, rollback.sh attempts to restore the previous application images without deleting the database volume. The operation is performed under the same environment lock as deployment and verification.

The rehearsal created a deliberately broken frontend that returned a healthy response at /health but returned HTTP 503 for /api. The smoke gate rejected it, rollback restored both original image IDs, and an authenticated smoke test passed. The rehearsal pipeline was successful because it correctly detected and recovered from the expected candidate failure.

5 DevOps Implementation continued

5.5.3 Recorded Recovery Evidence

```
PASS: PostgreSQL marker survived database container restart;
      probe row cleaned

Deployment failed; attempting rollback
Restored previous images for todo-staging

PASS: broken candidate failed its smoke gate; automatic rollback
      restored both original image IDs and the working API

PASS: BackendDown fired during the controlled staging outage

PASS: backend recovered, alert cleared and final authenticated
      smoke test passed
```

Figure 5.3 Excerpts from the recorded reports/rehearsal.txt artifact of develop build 5. Long lines have been wrapped for readability.

5.5.4 Interpreting the Result

A failing deployment and a successful recovery test are different outcomes. The candidate deployment returned failure and invoked rollback. The surrounding rehearsal expected that failure and verified that recovery actually occurred, so the rehearsal build completed successfully. This result does not represent the defective candidate as a successful release.

Database persistence, application-image rollback and monitoring recovery test different parts of the system. Their combined evidence demonstrates continuity of application data, recovery of a working frontend/API combination and restoration of monitoring availability.

5.5.5 Recovery Limits

Rollback requires previously running images and a compatible database schema. It does not undo irreversible data changes. Production and staging are separated, and the deliberately destructive candidate is restricted to the staging rehearsal.

5 DevOps Implementation continued

5.6 Experiment 6 Jira Workflow

Jira provides the project-management view of Helpdesk. The active HDEV board records the implementation stories under a sprint. Eight stories were moved to Done after their corresponding verification, while HDEV-10 remains In Progress for the student review and presentation.

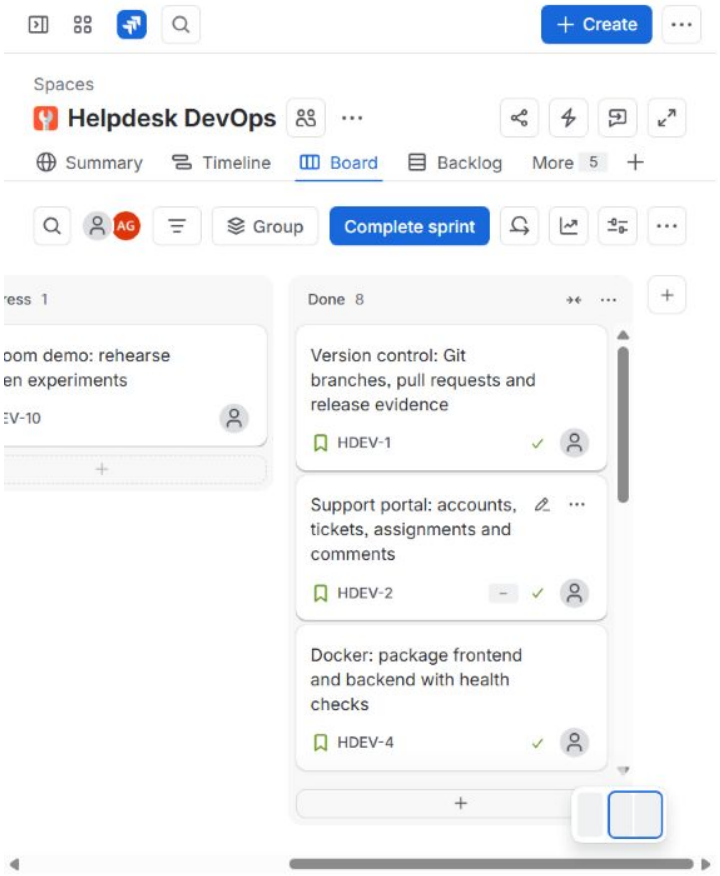


Figure 5.4 Actual Jira board showing completed implementation work and presentation progress.

Board: https://atharvagokhale.atlassian.net/jira/software/projects/HDEV/boards/36

HDEV-9 records acceptance criteria, Atharva Gokhale as assignee, two story points and parent epic HDEV-3. The project uses To Do, In Progress and Done. Portal tickets do not automatically create or synchronize Jira issues, and no historical burndown is fabricated.

5 DevOps Implementation continued

5.7 Experiment 7 Continuous Monitoring

The backend uses the Prometheus Python client to expose http_requests_total and http_request_duration_seconds metrics. Labels identify the method, matched route and response status where applicable. The /metrics endpoint is scraped by Prometheus, while cAdvisor supplies the available host/container resource measurements.

```
# Short intervals keep an academic demo responsive.
global:
  scrape_interval: 10s
  evaluation_interval: 10s
# Alerts are visible in Prometheus; delivery needs a separate Alertmanager.
rule_files:
  - /etc/prometheus/alert.rules.yml
# Service names resolve on the shared Compose network.
scrape_configs:
  - job_name: backend
    metrics_path: /metrics
    static_configs:
      - targets: ['backend:8000']
  - job_name: cadvisor
    static_configs:
      - targets: ['cadvisor:8080']
  - job_name: prometheus
    static_configs:
      - targets: ['localhost:9090']
```

Figure 5.5 Actual Prometheus scrape configuration. All three targets were verified UP.

5.7.1 Queries and Alert

The Grafana dashboard displays five measurements: CPU usage, memory, API request rate, p95 request latency and backend scrape availability. BackendDown checks backend scrape failure for one minute. The staging rehearsal verified that the alert fired during an outage and cleared after the backend recovered.

Prometheus alert state is available in its interface. External delivery through Alertmanager, email or SMS is not configured. Prometheus and Grafana implement the selected monitoring experiment; Nagios, MLflow and Airflow are not part of this project.

5 DevOps Implementation continued

5.7.2 Grafana Dashboard Evidence



Figure 5.6 Actual provisioned Grafana dashboard connected to Prometheus.

Dashboard: http://localhost:3000/d/devops-overview/devops-pipeline-overview

Five dashboard queries returned data during verification. Docker Desktop exposed root cgroup CPU and memory rather than individual container series. The two resource panels explicitly label the Docker host total fallback; request rate, latency and availability come from the backend scrape.

6 Testing

6.1 Test Strategy

The project combines isolated automated tests with runtime checks. pytest exercises application logic using temporary SQLite databases. Docker deployment tests exercise the real service network and PostgreSQL. The smoke gate checks the externally reachable application, while the recovery rehearsal introduces controlled failures in staging.

Test area	Coverage / evidence
Authentication	Registration, login, sessions, CSRF and logout/account boundaries.
Authorization	Requester isolation, forbidden staff actions and administrator-only agent creation.
Ticket workflow	Ticket CRUD, comments, assignments, status changes and dashboard counts.
Data relationships	Activity and comment behaviour, cascading ticket cleanup and temporary user cleanup.
Container integration	Image builds, service health and authenticated API operations through nginx.
Persistence	PostgreSQL marker survives container restart.
Recovery	Failed candidate rejected; original frontend/backend image IDs restored.
Monitoring	Three scrape targets UP; BackendDown firing and recovery snapshots.

6.2 Recorded Results

The archived JUnit report from develop build 5 records 32 tests with no failures. The smoke report and rehearsal artifact provide separate evidence of runtime success. These results support the tested behaviours; they are not a claim of exhaustive security, scalability or high-availability testing.

6.3 Reproduction

```
cd backend
python -m pytest tests -q --junitxml=../reports/unit-tests.xml
# Runtime check from repository root
python scripts/smoke.py http://localhost:8081
```

6 Testing continued

6.4 Verification Matrix

Check	Observed result	Evidence
Backend tests	32 passed	unit-tests.xml in develop build 5.
Production deployment	Successful recorded build	main build 4.
Staging recovery	Successful rehearsal	develop build 5.
Dashboard update	Successful updated staging build	develop build 6.
HTTP frontend health	200 for production and staging	Local runtime verification.
Prometheus	backend, cadvisor, prometheus UP	Targets API response.
Grafana	Five queries return series	dashboard-queries.json.
Jira	Eight Done; one In Progress	Captured live board.

6.5 Evidence Interpretation

Build numbers refer to the job and branch stated in the table; they are retained evidence runs rather than a claim that they will remain the latest builds. main and develop have independent build sequences. A source commit identifies the code used by a build, while runtime reports verify the behaviour after deployment.

The alert-firing.json artifact records the firing BackendDown alert. The recovery snapshot records the subsequent cleared state. A green target after recovery alone would not prove that the outage alert had fired, which is why both states were preserved.

6.6 Limitations of Verification

The project does not claim a public webhook, automatic Jira synchronization, email delivery, Nagios installation, a historical Scrum burndown or per-container resource data where the exporter only provides host totals. ShellCheck was unavailable during the recorded verification; syntax checks and executed container rehearsals provide the reported evidence.

7 Implementation

7.1 Website Overview

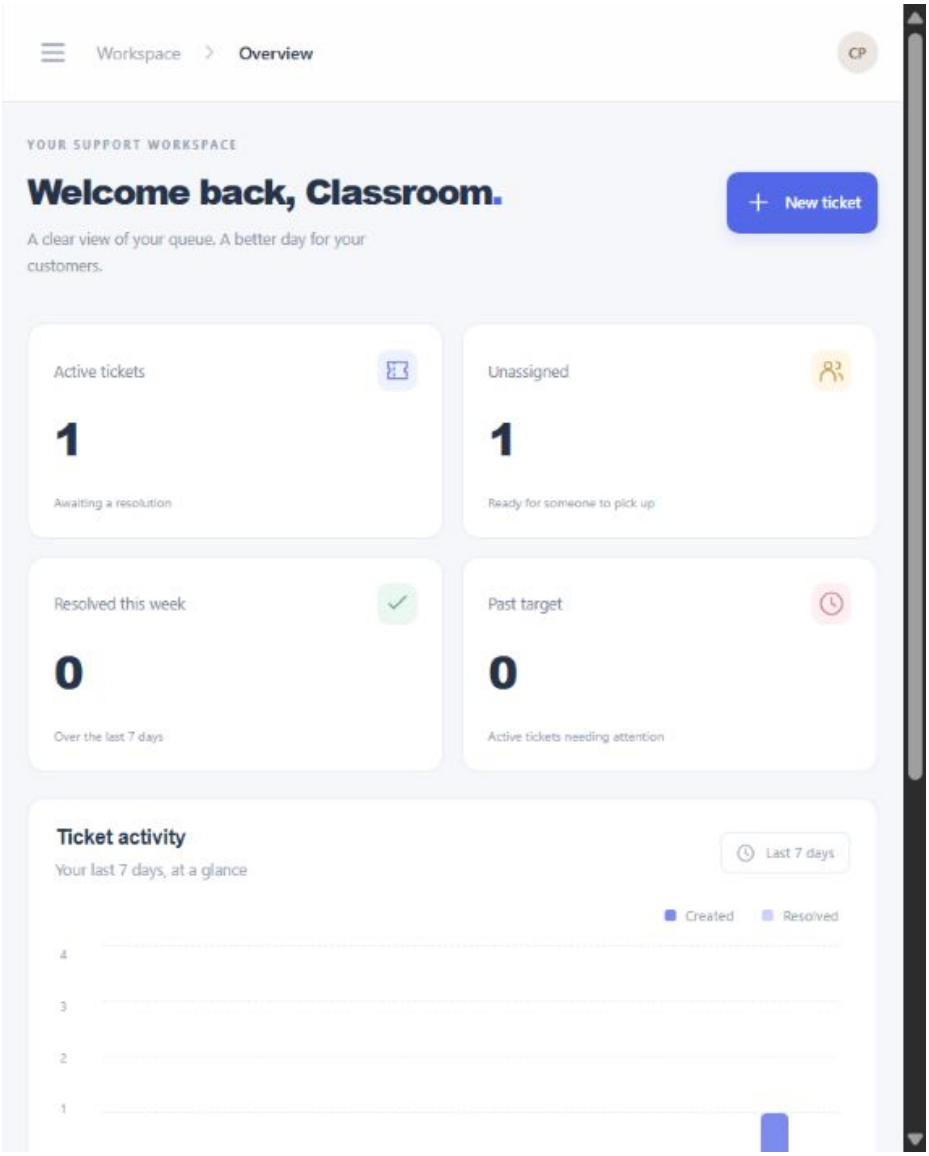


Figure 7.1 Actual production Helpdesk overview for the classroom administrator.

The overview summarizes the application queue. Its numbers are calculated from support ticket data. These product statistics are distinct from Grafana infrastructure and HTTP performance metrics.

7 Implementation continued

7.2 Ticket Detail Workflow

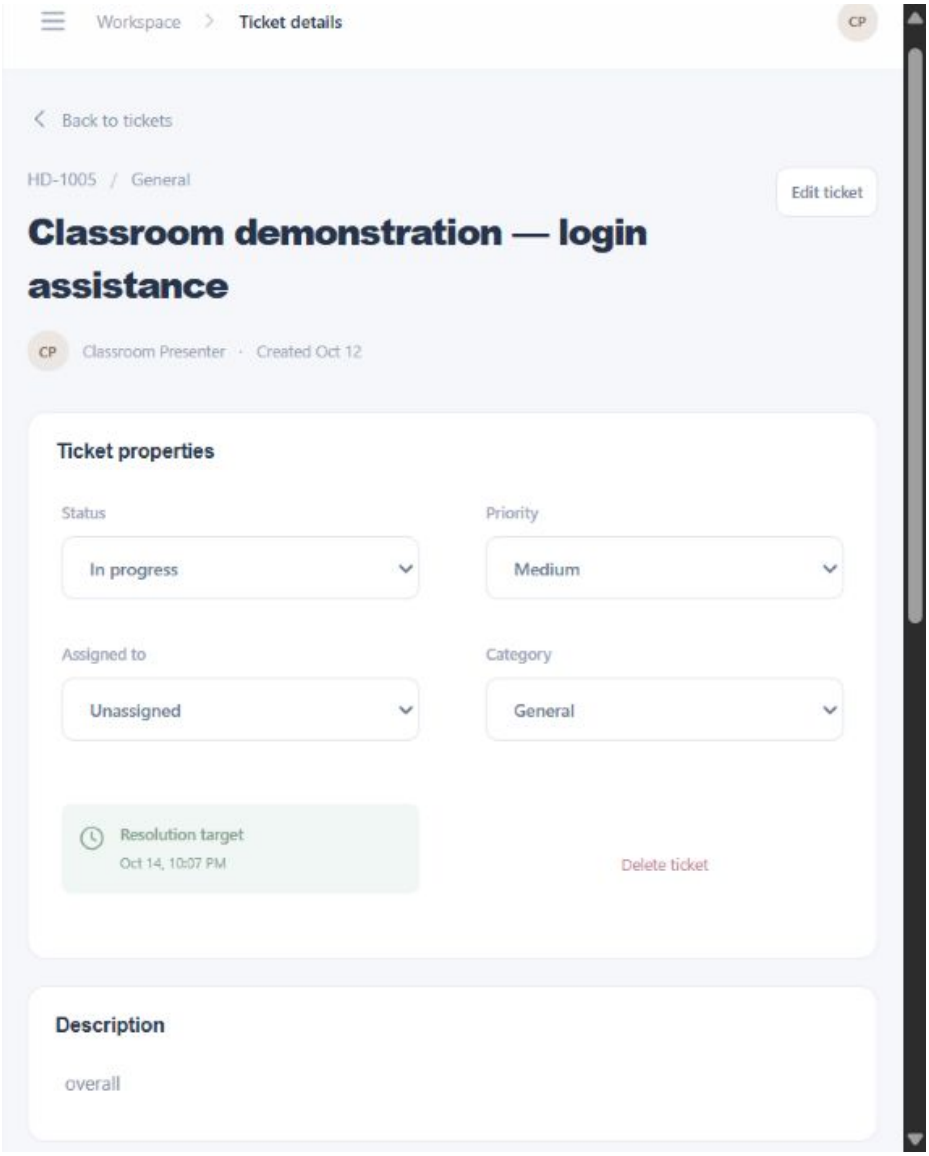


Figure 7.2 Actual classroom demonstration ticket with conversation, history and properties.

The ticket detail interface connects a request to its status, priority, category, assignee and activity. The captured ticket was created for the classroom demonstration. The user can open the same production site at http://localhost:8080.

8 Results and Discussion

8.1 Results

Area	Result
Application	React/Flask/PostgreSQL support portal with authenticated requester and staff workflows.
Git	Repository, deployment branches, merged PRs, conflict exercise and release tag.
Docker and Compose	Separate application images, coordinated services and persistent environment-specific databases.
Jenkins CI	Source-controlled pipeline with 32-test report and archived evidence.
Automated delivery	Polling, branch-based deployment, authenticated smoke gate and verified rollback.
Jira	Actual epic, nine stories, active sprint, statuses and an acceptance-criteria example.
Monitoring	Three scrape targets, five dashboard queries and tested BackendDown recovery.

8.2 Discussion

The project demonstrates that source control, testing, packaging, deployment and monitoring can be connected around one application. GitHub identifies the source, Jenkins executes the delivery process, Docker provides the running services and Prometheus/Grafana expose operational behaviour. Jira records the development work and its completion criteria.

The recovery rehearsal is particularly useful because it checks more than the presence of configuration files. A candidate that looked healthy at the frontend still failed API validation; the system then restored the prior images and verified a working application. This demonstrates why a meaningful smoke gate and preserved recovery evidence are necessary.

The implementation remains a local academic system. A production service would require stronger operational controls, backups, external alert delivery, public HTTPS and a broader performance and security review.

9 Conclusion and Future Scope

9.1 Conclusion

Helpdesk combines a practical support ticket website with a demonstrable Agile and DevOps workflow. Git and GitHub provide source history and change traceability. Docker packages the application, Compose coordinates its services, and Jenkins automates tests, image builds and branch-specific delivery. The application is checked after deployment rather than assuming that container startup guarantees success.

The recorded staging rehearsal verifies PostgreSQL persistence, rejection of a broken candidate, recovery of the previous application images and restoration of backend monitoring. Jira connects the work to a sprint and acceptance criteria, while Prometheus and Grafana provide a separate operational view of the application.

9.2 Future Scope

The portal can be extended with email notifications, password reset, attachments, richer service-level reporting and a formal approval workflow. Delivery improvements include a secure webhook when a suitable endpoint is available, image registry publication, dependency/security scanning, environment approvals and cloud hosting.

Operational improvements include scheduled database backups and restore drills, external alert routing, broader performance tests, multi-worker metrics support and reliable per-container resource monitoring. Automatic Jira development integration could associate repository events with work items; synchronization of end-user support tickets would be a separate feature.

9.3 Learning Outcome

The project provides practical experience in managing a source repository, packaging and orchestrating services, building a CI/CD pipeline, validating runtime behaviour, recovering from failures, tracking Agile work and interpreting monitoring data. The seven experiments can be presented as one lifecycle with evidence at each connection.

A Appendix Running the Project

A.1 Existing Classroom Deployment

Keep Docker Desktop running. Open production on localhost:8080 or staging on localhost:8081. Jenkins, Prometheus, Grafana and cAdvisor use the links on the following page. Localhost addresses work on the computer running the containers.

A.2 Fresh Environment

```
git clone https://github.com/gokhale-atharva/helpdesk-devops.git
cd helpdesk-devops
python scripts/init-env.py
docker compose -p todo-production --env-file .env.production config --quiet
docker compose -p todo-production --env-file .env.production up -d --build --wait
python scripts/smoke.py http://localhost:8080
```

The repository is private, so cloning requires an authorized account. init-env.py prepares private configuration. For first-run Jenkins setup, follow docs/automated-setup.md. The supplied deployment already has a presenter account; for a fresh database, create an administrator interactively using backend/manage.py inside the backend container.

A.3 Administrator and Service Inspection

```
docker compose -p todo-production --env-file .env.production exec backend python manage.py
docker compose -p todo-production --env-file .env.production ps
docker compose -p todo-staging --env-file .env.staging ps
```

A.4 Controlled Recovery Demonstration

In Jenkins, open helpdesk → develop → Build with Parameters, enable REHEARSE_STAGING and start the build. The exercise deliberately interrupts staging and deploys a defective candidate before restoring the working version. For a short presentation, use the retained successful build 5 and its reports instead of repeating the outage live.

A Appendix Demonstration Links

Purpose	Address
Production	http://localhost:8080
Staging	http://localhost:8081
GitHub	https://github.com/gokhale-atharva/helpdesk-devops
Jenkins jobs	http://localhost:8088/job/helpdesk/
Recovery evidence	http://localhost:8088/job/helpdesk/job/develop/5/
Prometheus targets	http://localhost:9090/targets
Prometheus alerts	http://localhost:9090/alerts
Grafana	http://localhost:3000/d/devops-overview/devops-pipeline-overview
cAdvisor	http://localhost:8082
Jira board	https://atharvagokhale.atlassian.net/jira/software/projects/HDEV/boards/36

A.5 Access and Credentials

The classroom portal account is presenter@helpdesk.local. Jenkins uses helpdesk-admin, and Grafana uses admin. Passwords are stored in the separate private presentation login guide and are intentionally excluded from this submission. GitHub and Jira use existing account authentication. Prometheus and cAdvisor do not require a login in this loopback-only lab.

A.6 Project Evidence Sources

Implementation references: backend/portal.py, backend/tests/, frontend/, Jenkinsfile, docker-compose.yml, docker-compose.monitoring.yml, scripts/deploy.sh, scripts/rollback.sh, scripts/rehearse.sh and monitoring/. Runtime evidence: Jenkins develop build 5 artifacts, the recorded dashboard query results, the Git conflict transcript and actual interface screenshots reproduced in this report.

Presentation sequence: website → GitHub → Docker/Compose → Jenkins tests and deployment → Jira board → Prometheus and Grafana.